



Builder Design Pattern

Prepared BY

Team The Axis (Team 6)

University of Barishal

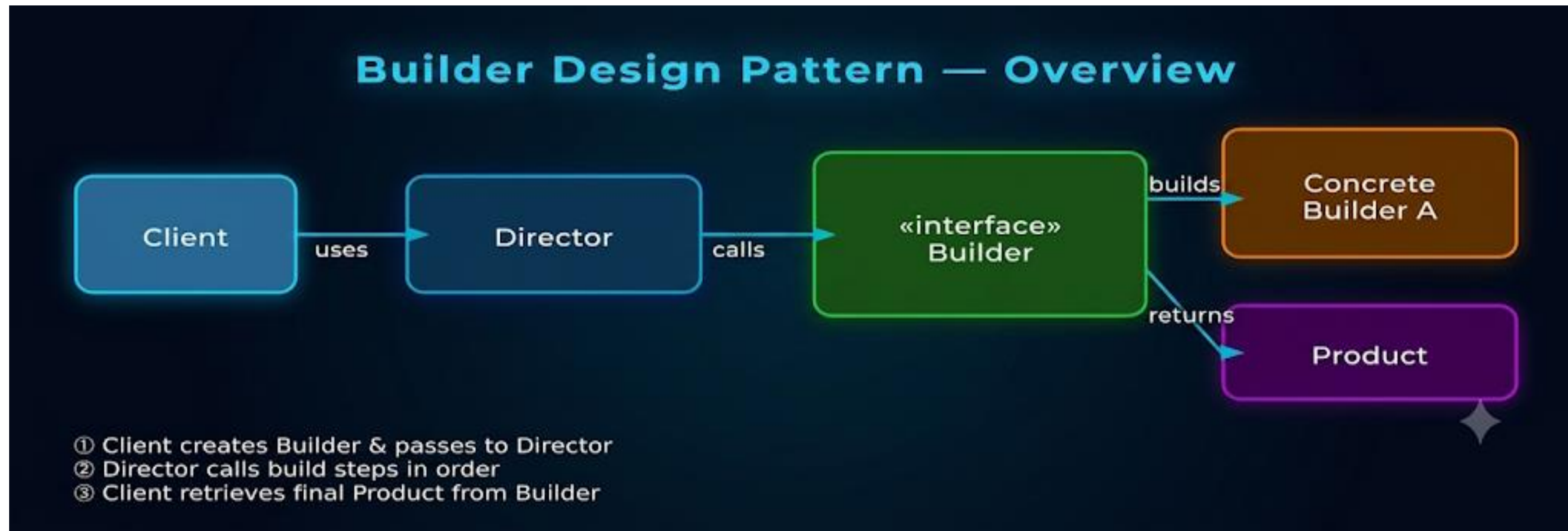
Instructor

Md Samsuddoha

Assistant Professor
University of Barishal

What is the Builder Pattern?

The Builder Pattern is a **Creational Design Pattern** that constructs complex objects step by step. It separates the construction process from the object's representation, allowing the same construction process to create different types and representations of an object.



Key Features

- ✓ Step-by-step object creation
- ✓ Separation of construction and representation
- ✓ Supports multiple representations
- ✓ Improves readability and maintainability

Intent of Builder Pattern

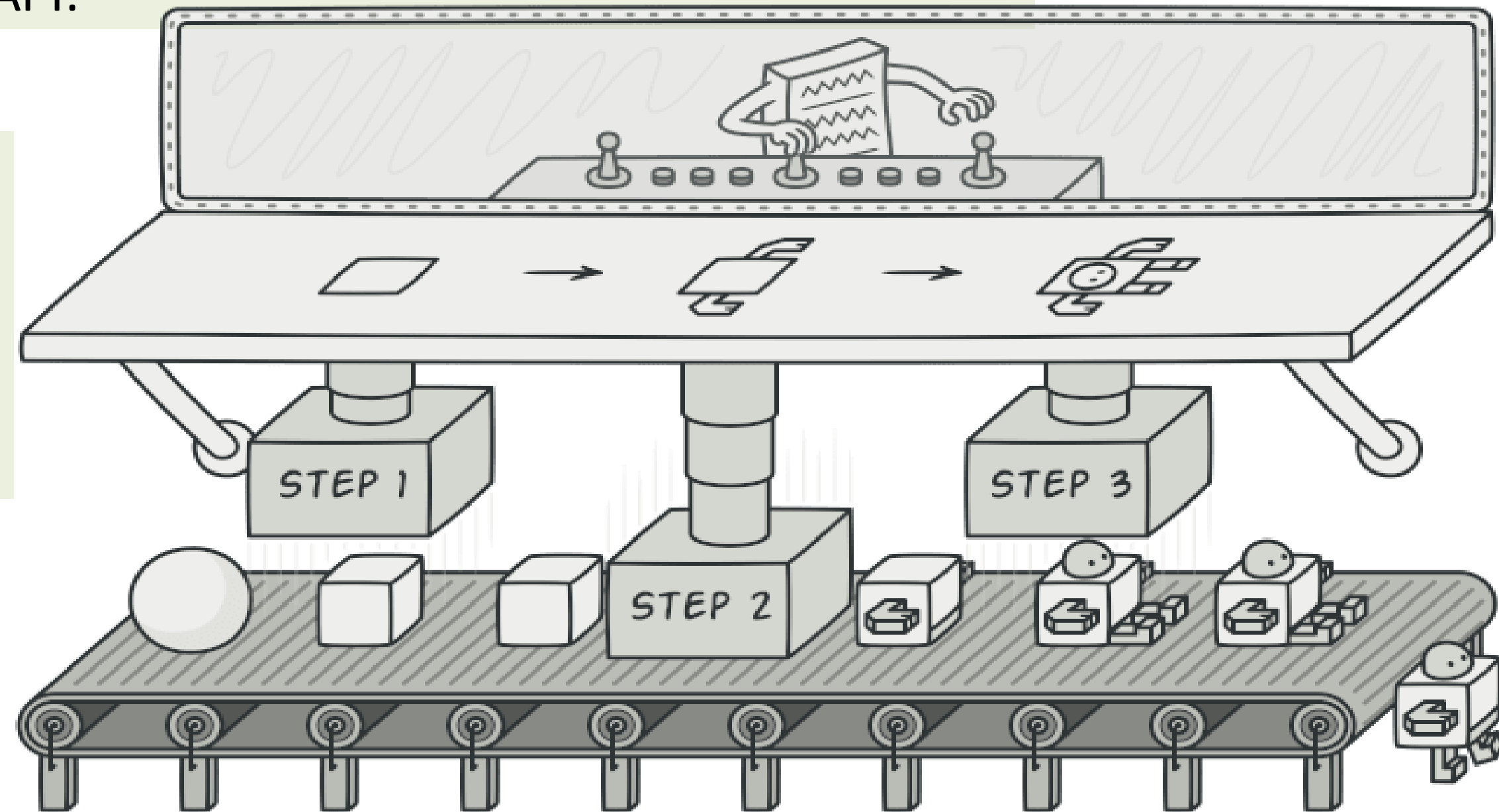
The Builder Pattern aims to:

- ✓ Construct complex objects step by step.
- ✓ Use the same construction process to create different representations.
- ✓ Encapsulate object construction.
- ✓ Avoid **telescoping constructors** (constructors with 10+ parameters)
- ✓ Provide a clear and flexible object creation API.

Example:

```
Car car = carBuilder.getProduct();  
Manual manual = manualBuilder.getProduct();
```

Same construction process → Different outputs.



Builder Pattern — Why We Need It

1

Too many constructor parameters

Avoids constructors with 8-10 optional parameters that are error-prone and hard to read.

2

Step-by-step construction

Builds complex objects piece by piece, in a controlled sequence.

3

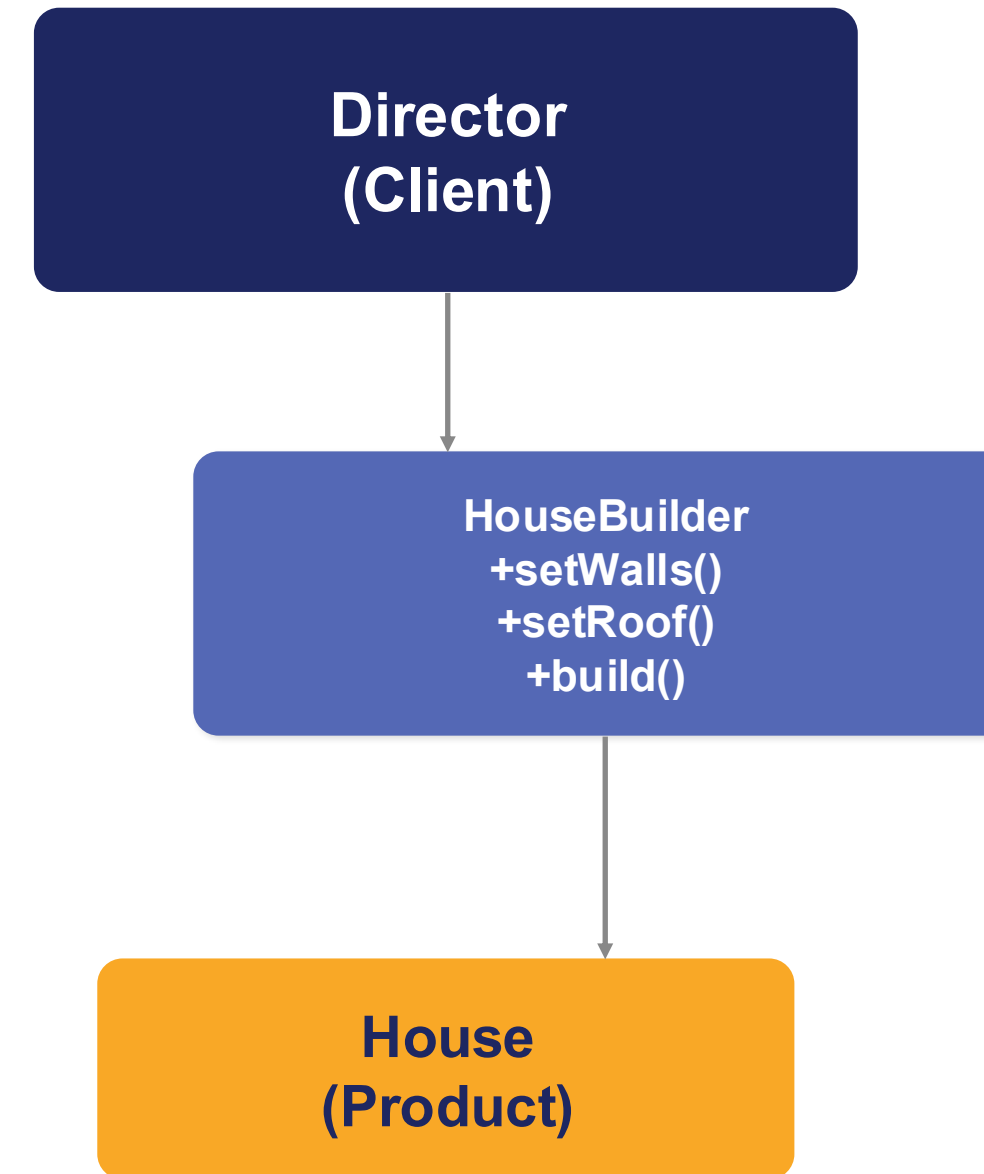
Immutability

Produces a fully-formed, immutable object only when build() is called.

4

Readable client code

Chained, fluent calls describe exactly what is being configured.



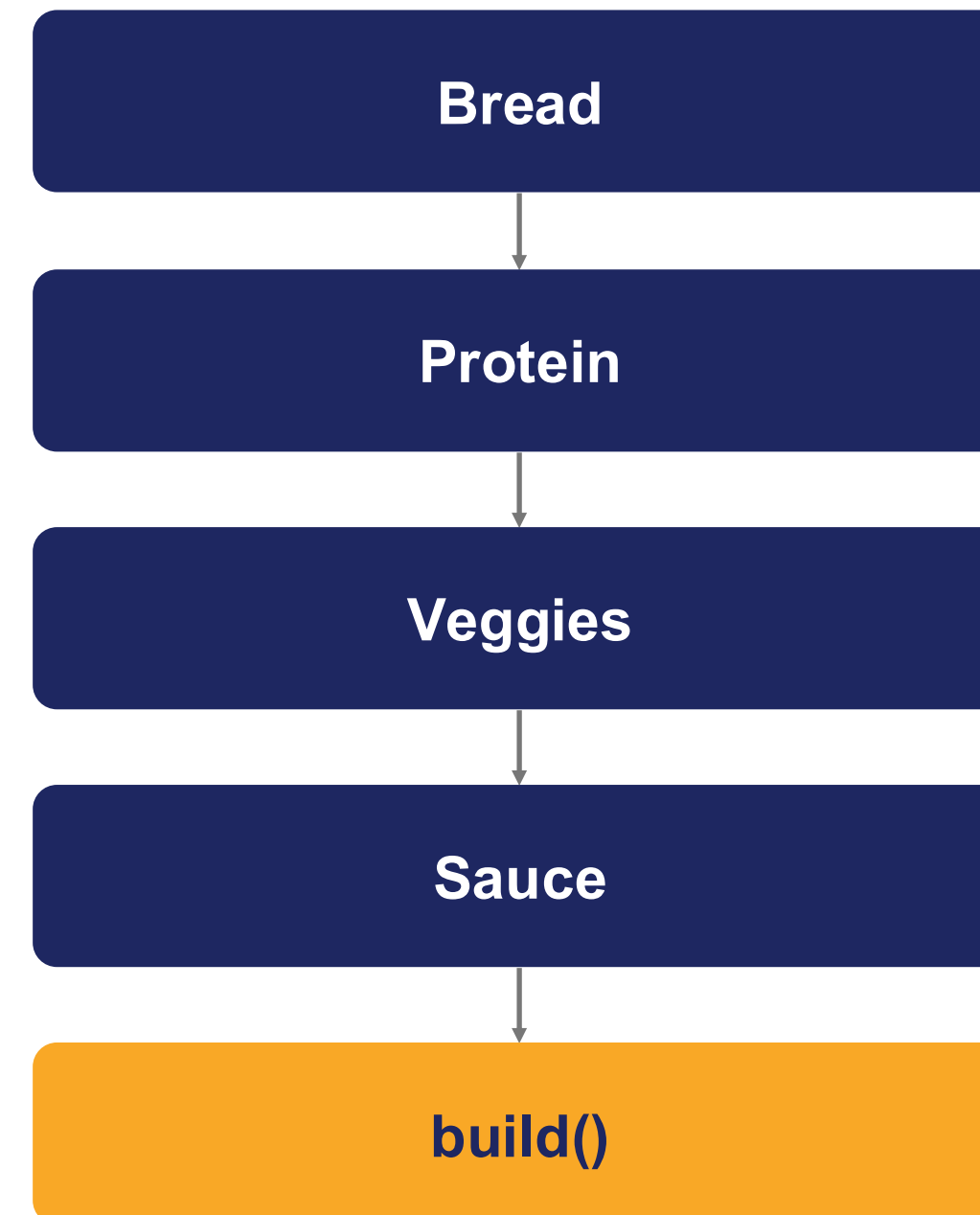
Caption: A Director calls step methods on a Builder, which assembles the final Product object.

Builder — Real-Life Example

Custom Sandwich Order (Subway-style)

Customer builds the order step by step:

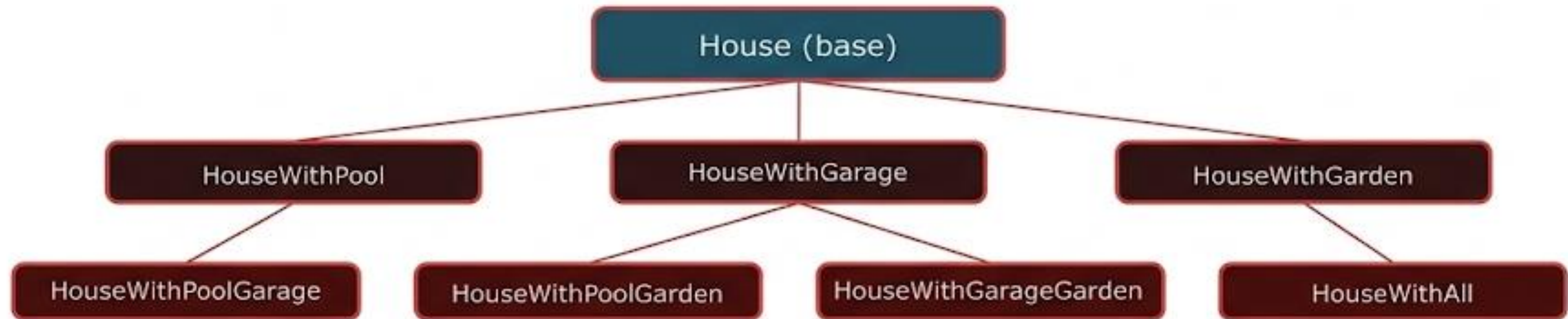
- Choose bread → `setBread("Wheat")`
- Add protein → `setProtein("Chicken")`
- Add veggies → `addVeggies([...])`
- Add sauce → `setSauce("Mayo")`
- Finish → `build()` returns finished Sandwich



Result: a fully assembled Sandwich object — no giant constructor needed.

Problem: Subclass Explosion

Problem 1: Subclass Explosion



⚠ Every new feature multiplies the number of subclasses!

Scenario: To build a House object with various optional features (pool, garage, garden, fireplace, etc.), you would need a separate subclass for every possible combination. With just 4 features, that is already $2^4 = 16$ classes. Adding one more doubles that! ✨

Problem: Telescoping Constructor

Problem 2: Telescoping Constructor

```
class House {  
    House(int rooms)  
    House(int rooms, boolean pool)  
    House(int rooms, boolean pool, boolean garage)  
    House(int rooms, boolean pool, boolean garage, String roof)  
    House(int rooms, boolean pool, boolean garage,  
          String roof, boolean garden, int floors)  
    // ... dozens more overloads ...  
}
```

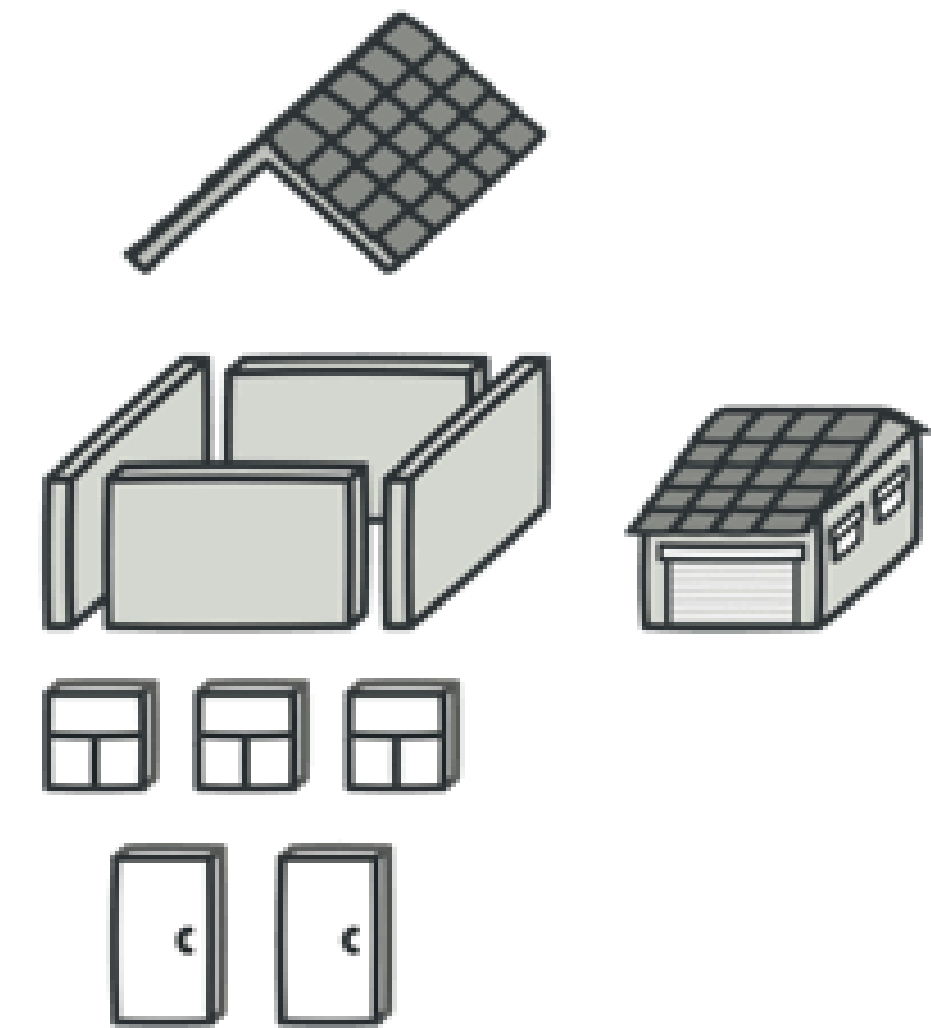
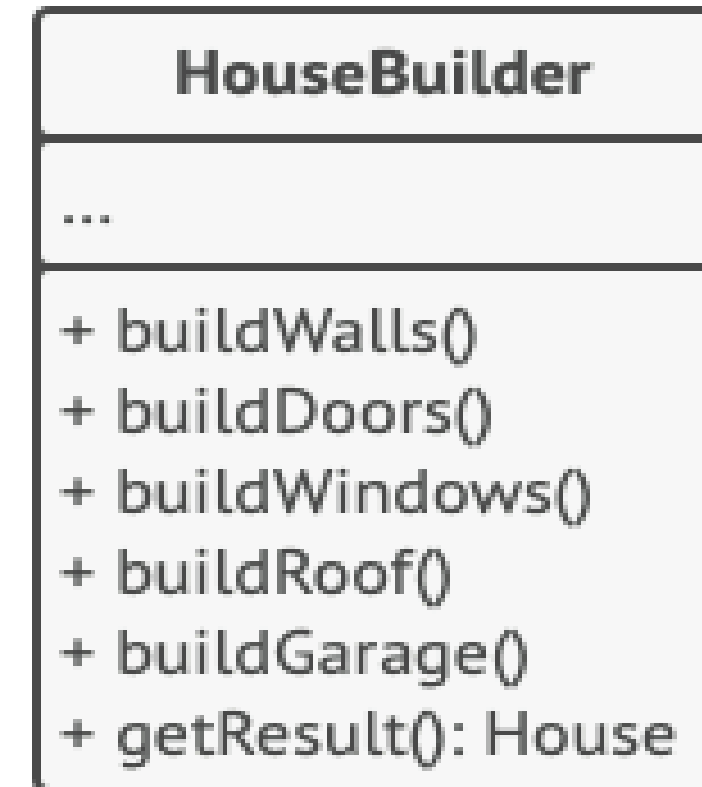


⚠ Most parameters are unused most of the time — ugly, error-prone calls!
e.g. `new House(4, false, true, null, false, 0, null, false, null);`

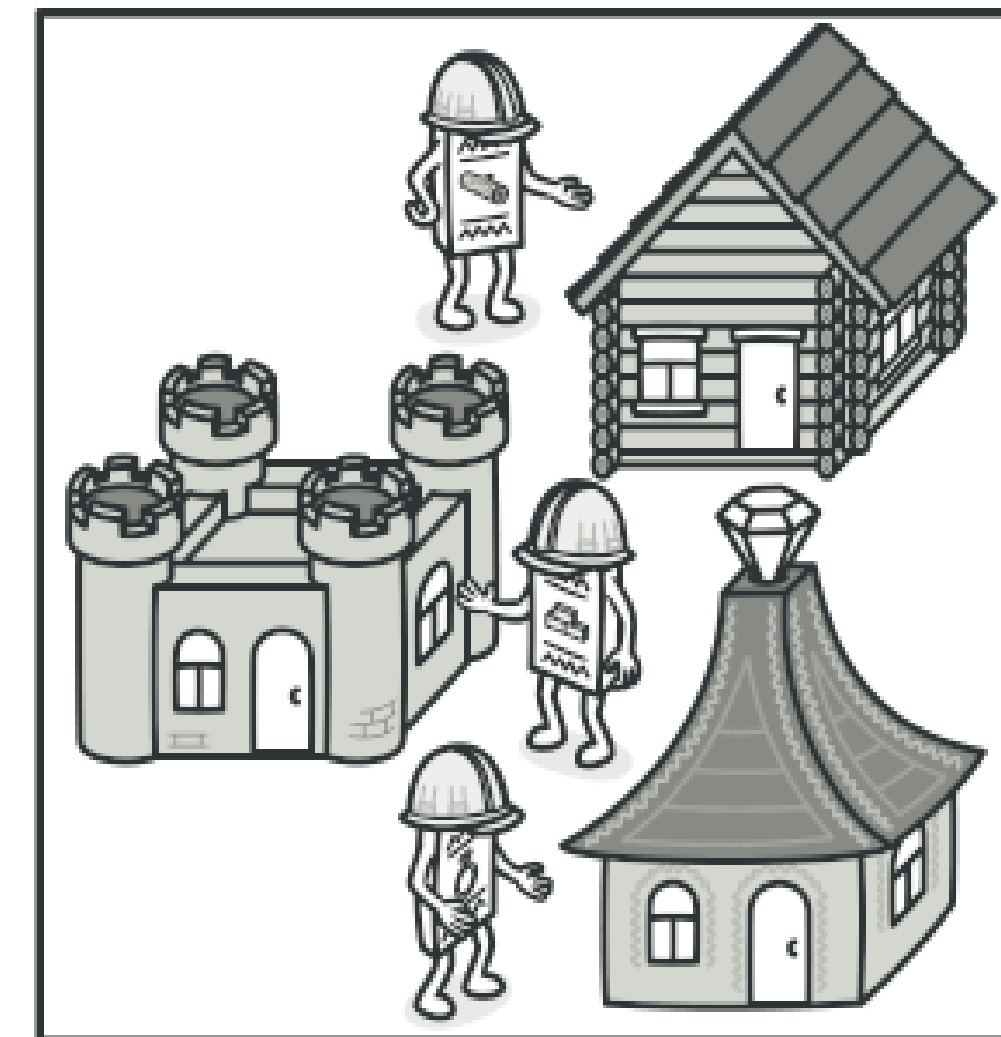
Anti-Pattern: Instead of subclasses, you create one giant constructor with all possible parameters. Most are null or default in most calls. The code becomes unreadable, and callers must remember the order of parameters. This is the “Telescoping Constructor” anti-pattern.

Solution

1. Extract object construction into Builder classes.
2. Build objects step by step.
3. Configure only necessary features.
4. Hide construction details from the client.
5. Improve flexibility and maintainability.



1. Client specifies the required configuration.
2. Builders construct the object step by step.
3. Different builders implement the same steps differently.
4. Same construction process produces different products.



Director

Role of the Director

Defines the order of building steps to create a consistent product.
Delegates how each step is done to the Builder.

Workflow

Client assigns a Builder to the Director → Director executes steps in fixed order → Client gets final product from Builder.

Key Benefits

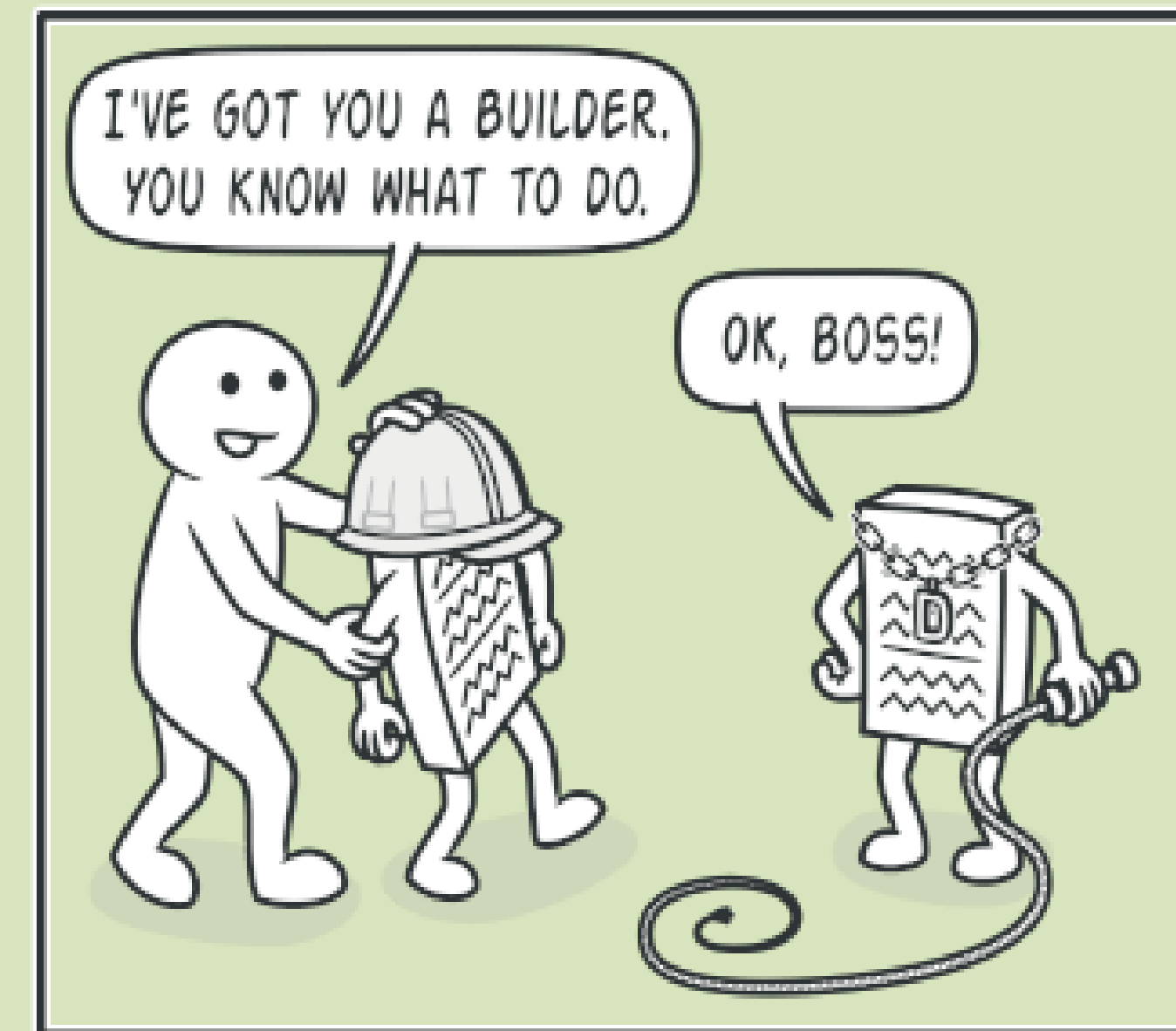
- Reusable construction logic
- Hides product creation details from client
- Centralizes complex build process

Note: Director is optional but improves code organization and reusability.

Example:

```
class Director {  
    void construct(HouseBuilder builder) {  
        builder.buildFoundation();  
        builder.buildWalls();  
        builder.buildRoof();  
    }  
}
```

→ It controls the order of steps (core Director role)



Without Builder Pattern

Problem 1: Constructor Explosion

- Too many constructors are needed.
- As the number of optional parameters increases, the number of constructor combinations grows rapidly.

Problem 2: Readability Problem

- Cannot understand what true/false means
- No idea which boolean belongs to which feature

```
new Laptop("i7", 16, 512, true, false, "Black")
```

↑ ↑
SSD or Graphics Card?

- Parameter meanings are unclear.
- Developers must check the constructor every time.

Problem 3: No Validation

- No checking before object creation

```
BankAccount acc = new BankAccount("", -500, "Savings");
```
- Negative balance or null values create invalid objects

Problem 4: Mutable Object

- Object data can be changed after creation
- Anyone can modify using setter methods

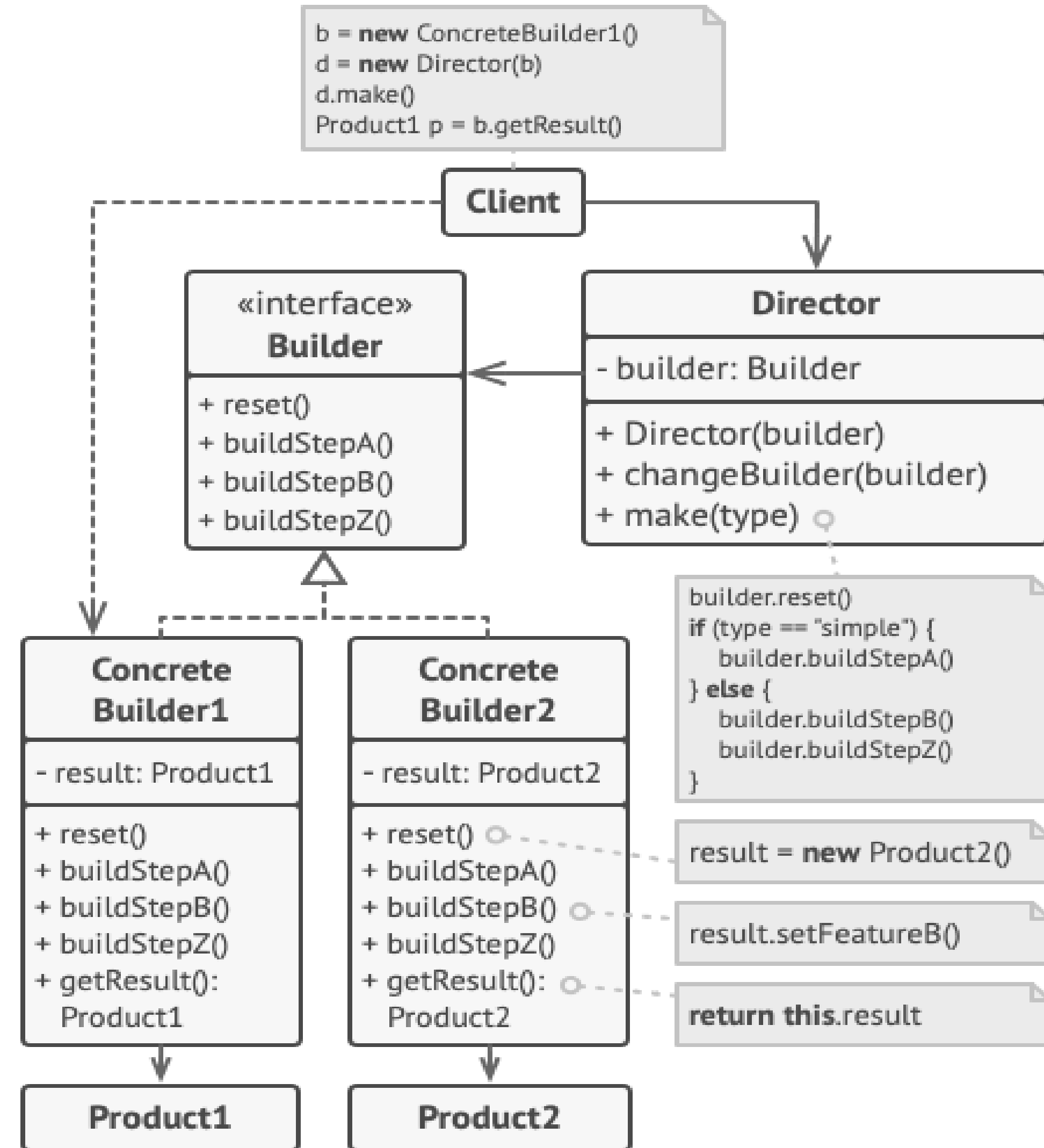
```
Student s = new Student("Rahim", 101, "A+");  
s.setName("Karim");  
s.setGrade("F");
```

Problem 5: Scalability Problem

- Hard to add new features
- Adding one parameter = changing all constructors

Structure

1. The **Builder** interface declares product construction steps that are common to all types of builders.
2. **Concrete Builders** provide different implementations of the construction steps. Concrete builders may produce products that don't follow the common interface.
3. **Products** are resulting objects. Products constructed by different builders don't have to belong to the same class hierarchy or interface.
4. The **Director** class defines the order in which to call construction steps, so we can create and reuse specific configurations of products.
5. **Client** must attach a builder to the director (usually via constructor) and the director uses it for construction; alternatively, the builder can be passed to the director's method each time, allowing different builders for different products.



PizzaBuilder.java

```
1 // BUILDER INTERFACE - Declares all construction steps
2 public interface PizzaBuilder {
3     void reset();                // Reset builder state
4     void buildSize(String size); // Build pizza size
5     void buildCheese();          // Add cheese topping
6     void buildPepperoni();       // Add pepperoni topping
7     void buildMushroom();        // Add mushroom topping
8     void buildOlive();           // Add olive topping
9     void buildSauce(String sauce); // Add sauce
10    Object getResult();           // Return final product
11 }
```

RegularPizza.java

```
ider > RegularPizza.java > RegularPizza
1 // PRODUCT 1 - Regular Pizza (First product type)
2 public class RegularPizza {
3     // Private fields
4     private String size;
5     private boolean cheese;
6     private boolean pepperoni;
7     private boolean mushroom;
8     private boolean olive;
9     private String sauce;
10
11     // Setter methods
12     public void setSize(String size) { this.size = size; }
13     public void setCheese(boolean cheese) { this.cheese = cheese; }
14     public void setPepperoni(boolean pepperoni) { this.pepperoni = pepperoni; }
15     public void setMushroom(boolean mushroom) { this.mushroom = mushroom; }
16     public void setOlive(boolean olive) { this.olive = olive; }
17     public void setSauce(String sauce) { this.sauce = sauce; }
18
19     // Display product details
20     public void show() {
21         System.out.println("Regular Pizza: " + size +
22             ", Cheese:" + cheese +
23             ", Pepperoni:" + pepperoni +
24             ", Mushroom:" + mushroom +
25             ", Olive:" + olive +
26             ", Sauce:" + sauce);
27     }
28 }
```

VegPizza.java

```
1 // PRODUCT 2 - Veg Pizza (Different product, different hierarchy)
2 public class VegPizza {
3     // Private fields (no pepperoni - different from RegularPizza)
4     private String size;
5     private boolean cheese;
6     private boolean mushroom;
7     private boolean olive;
8     private String sauce;
9
10    // Setter methods
11    public void setSize(String size) { this.size = size; }
12    public void setCheese(boolean cheese) { this.cheese = cheese; }
13    public void setMushroom(boolean mushroom) { this.mushroom = mushroom; }
14    public void setOlive(boolean olive) { this.olive = olive; }
15    public void setSauce(String sauce) { this.sauce = sauce; }
16
17    // Display product details
18    public void show() {
19        System.out.println("Veg Pizza: " + size +
20            ", Cheese:" + cheese +
21            ", Mushroom:" + mushroom +
22            ", Olive:" + olive +
23            ", Sauce:" + sauce);
24    }
25 }
```

RegularPizzaBuilder.java

```
// CONCRETE BUILDER 1 - Builds Regular Pizza
public class RegularPizzaBuilder implements PizzaBuilder {
    private RegularPizza result; // Holds the product being built
    // Constructor - initializes builder
    public RegularPizzaBuilder() {
        reset();
    }
    @Override
    public void reset() {
        this.result = new RegularPizza(); // Create new product
    }
    @Override
    public void buildSize(String size) {
        result.setSize(size);
    }
    @Override
    public void buildCheese() {
        result.setCheese(cheese: true);
    }
    @Override
    public void buildPepperoni() {
        result.setPepperoni(pepperoni: true);
    }
    @Override
    public void buildMushroom() {
        result.setMushroom(mushroom: true);
    }
    @Override
    public void buildOlive() {
        result.setOlive(olive: true);
    }
    @Override
    public void buildSauce(String sauce) {
        result.setSauce(sauce);
    }
    @Override
    public Object getResult() {
        return result; // Return final product
    }
}
```

VegPizzaBuilder.java

```
// CONCRETE BUILDER 2 - Builds Veg Pizza (Different product)
public class VegPizzaBuilder implements PizzaBuilder {
    private VegPizza result; // Holds different product type
    // Constructor - initializes builder
    public VegPizzaBuilder() {
        reset();
    }
    @Override
    public void reset() {
        this.result = new VegPizza(); // Create new veg pizza
    }
    @Override
    public void buildSize(String size) {
        result.setSize(size);
    }
    @Override
    public void buildCheese() {
        result.setCheese(cheese: true);
    }
    @Override
    public void buildPepperoni() {
        // PROBLEM: Veg pizza cannot have pepperoni
        System.out.println(x: "No pepperoni for Veg Pizza");
    }
    @Override
    public void buildMushroom() {
        result.setMushroom(mushroom: true);
    }
    @Override
    public void buildOlive() {
        result.setOlive(olive: true);
    }
    @Override
    public void buildSauce(String sauce) {
        result.setSauce(sauce);
    }
    @Override
    public Object getResult() {
        return result; // Return different product type
    }
}
```

PizzaDirector.java

```
// DIRECTOR - Defines the order of construction steps
public class PizzaDirector {
    private PizzaBuilder builder; // Current builder being used
    // Constructor - attaches builder (as shown in diagram: d = new Director(b))
    public PizzaDirector(PizzaBuilder builder) {
        this.builder = builder;
    }
    // Change builder at runtime (as shown in diagram: changeBuilder(builder))
    public void changeBuilder(PizzaBuilder builder) {
        this.builder = builder;
    }
    // Predefined configuration 1 - Margherita
    public void makeMargherita() {
        builder.reset(); // Step 1: Reset
        builder.buildSize(size: "Large"); // Step 2: Set size
        builder.buildCheese(); // Step 3: Add cheese
        builder.buildSauce(sauce: "Tomato"); // Step 4: Add sauce
        System.out.println(x: "Making Margherita Pizza");
    }
    // Predefined configuration 2 - Pepperoni
    public void makePepperoni() {
        builder.reset(); // Step 1: Reset
        builder.buildSize(size: "Medium"); // Step 2: Set size
        builder.buildCheese(); // Step 3: Add cheese
        builder.buildPepperoni(); // Step 4: Add pepperoni
        builder.buildSauce(sauce: "BBQ"); // Step 5: Add sauce
        System.out.println(x: "Making Pepperoni Pizza");
    }
    // Custom make method - like make(type) in diagram
    public void make(String type, String size, boolean cheese, boolean pepperoni,
        boolean mushroom, boolean olive, String sauce) {
        builder.reset(); // Reset first
        builder.buildSize(size); // Build size
        // Add toppings based on parameters
        if (cheese) builder.buildCheese();
        if (pepperoni) builder.buildPepperoni();
        if (mushroom) builder.buildMushroom();
        if (olive) builder.buildOlive();
        if (sauce != null) builder.buildSauce(sauce);

        System.out.println("Making " + type + " Pizza");
    }
    // Get final product
    public Object getResult() {
        return builder.getResult();
    }
}
```

PizzaSolution.java

```
// CLIENT - Attaches builder to director (as shown in diagram)
public class PizzaSolution {
    Run | Debug
    public static void main(String[] args) {
        System.out.println(x: "=== BUILDER PATTERN - CONSTRUCTOR EXPLOSION SOLVED ===\n");

        // Client creates concrete builder
        RegularPizzaBuilder builder1 = new RegularPizzaBuilder();

        // Client attaches builder to director via constructor
        PizzaDirector director = new PizzaDirector(builder1);

        // Director constructs the product
        director.makeMargherita();

        // Client gets result from builder
        RegularPizza pizza1 = (RegularPizza) director.getResult();
        pizza1.show();

        System.out.println();

        // Client can change builder at runtime
        VegPizzaBuilder builder2 = new VegPizzaBuilder();
        director.changeBuilder(builder2);
        director.makePepperoni();
        VegPizza pizza2 = (VegPizza) director.getResult();
        pizza2.show();

        System.out.println(x: "\n===== PROBLEM SOLVED! =====");
        System.out.println(x: " Only ONE constructor needed!");
        System.out.println(x: " No constructor explosion!");
        System.out.println(x: " Director controls construction order");
        System.out.println(x: " Different builders = different products");
        System.out.println(x: " Code follows the Builder Pattern structure");
    }
}
```

Output:

```
=== BUILDER PATTERN - CONSTRUCTOR EXPLOSION SOLVED ===
```

```
Making Margherita Pizza
```

```
Regular Pizza: Large, Cheese:true, Pepperoni:false, Mushroom:false, Olive:false, Sauce:Tomato
```

```
No pepperoni for Veg Pizza
```

```
Making Pepperoni Pizza
```

```
Veg Pizza: Medium, Cheese:true, Mushroom:false, Olive:false, Sauce:BBQ
```

```
===== PROBLEM SOLVED! =====
```

```
Only ONE constructor needed!
```

```
No constructor explosion!
```

```
Director controls construction order
```

```
Different builders = different products
```

```
Code follows the Builder Pattern structure
```

Pros

- ✓ We can construct objects step-by-step, defer construction steps, or run steps recursively.
- ✓ We can reuse the same construction code when building various representations of products.
- ✓ **Single Responsibility Principle:** We can isolate complex construction code from the business logic of the product.

Cons

The overall complexity of the code increases since the pattern requires creating multiple new classes.

When to use:

1. Object has 4+ optional parameters
2. Same construction steps → different representations
3. when need to hide product during construction
4. Telescoping constructor becoming unmanageable
5. When want fluent interface (method chaining)



References

Refactoring Guru — Mediator

refactoring.guru/design-patterns/mediator (main reference: concept, structure, pros & cons)

GeeksForGeeks

Design Patterns: Elements of Reusable Object-Oriented Software (GoF, 1994) — original definition



Thank You

Any Questions?